SIMATS ENGINEERING

Department of Artificial Intelligence & Machine Learning

ITA05 – Computer Vision

ASSIGNMENT 3 – CO3

Analyze, Compare and Justify Feature Detection and Matching Techniques for a Smart Self-Checkout at the College Canteen Store

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison.

STUDENT DETAILS

Particulars	Details	Particulars	Details
Student Name	M. Sahasra	Register Number	192425317
Department / Section	Artificial Intelligece & Machine Learning	Academic Year / Semester	2025-2026
Faculty Name	Dr. Bindu G	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	_____	Course Code	ITA05

Assignment Title: Analyze, Compare and Justify Feature Detection and Matching Techniques for a Smart Self-Checkout at the College Canteen Store (Total Marks – 50).

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Image Processing Types	4		
2	Image Preprocessing	4		
3	Image Representation	4		
4	Edge Detection	5		
5	Corner Detection	5		
6	Hough Transform	5		
7	Feature Matching	5		
8	PCA-Based Feature Reduction	4		
9	Integrated Self-Checkout Pipeline	6		
10	Multi-Item Recognition and System Adaptation	8		
TOTAL		50		

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Evidence item	What to include in final practical submission	Status	Notes
Dataset	At least 30 real snack packets, clean front-facing reference image for each, and query photographs under different conditions.		
Ground truth	Product IDs, manually marked packet outlines/corners, and labels for each query photograph.		
Source code	Commented Python/OpenCV/NumPy code, fixed parameters, reproducible file paths and execution instructions.		
Quantitative results	Top-1/top-5 accuracy, precision, recall, repeatability, corner error, time and memory with units.		
Visual evidence	Input photos, preprocessed images, edge/corner maps, Hough accumulator, feature matches and recognition rankings.		
Integrity	Clearly distinguish measured results, calculated values, estimates and illustrative examples. Do not invent experimental outcomes.		

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Stage	Evidence to retain	Used in tasks
1. Acquire	30+ packet photos; product labels; challenging examples	1–10
2. Prepare	Grayscale/colour normalization, resize, denoising and glare handling	2, 4, 7, 9, 10
3. Detect	Edges, corners, contours, Hough lines and keypoints	4–7
4. Represent	Pixel vectors, HSV histograms, ORB/SIFT descriptors and PCA projections	3, 7–9
5. Match	Descriptor distances, ratio test, homography and RANSAC inliers	7–9
6. Decide	Top-k ranking, confidence threshold, reject/confirm rule and price lookup	1, 9, 10
7. Validate	Accuracy, runtime, memory, failure cases and design decision table	All

SUBMISSION CHECKLIST (not separately marked)

- ☐ All ten tasks are answered in the given order and all specified candidate techniques are compared.
- ☐ Use the same labelled packet dataset for fair comparisons wherever required.
- ☐ All numerical values are labelled as measured, calculated, estimated or illustrative.
- ☐ Feature matching uses a defined distance metric and a documented acceptance rule.
- ☐ Recognition accuracy is computed using ground-truth product IDs.
- ☐ Processing time is measured on the stated CPU-only hardware.
- ☐ Memory estimates include descriptors, PCA matrices and indexing overhead where applicable.
- ☐ Incorrect or uncertain matches are rejected or sent for cashier confirmation rather than billed automatically.
- ☐ Figures, code outputs and tables are numbered and captioned.

COMPUTER VISION

ASSIGNMENT 3 – CO3

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

ASSIGNMENT 3 – CO3: FEATURE DETECTION AND MATCHING FOR SMART SELF-CHECKOUT

Scenario – Smart Self-Checkout at the College Canteen Store

The college canteen store has long queues at the billing counter during breaks. It wants a simple "Smart Self-Checkout": a student holds a snack packet (biscuits, chips, chocolate, juice box, noodles) in front of a counter camera, and the program recognizes the product by comparing the photo with the store's product image database, then adds its price to the bill. Real photos are not perfect: the packet may be tilted, rotated, close to or far from the camera, crushed or folded, partly covered by fingers, slightly blurred, or show glare from its shiny plastic. The program must extract features from each image, represent them compactly, and compare (match) the photo with the database to bill the right product.

You are the vision engineer. In every task, three candidate techniques are given. You must apply all three, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

The canteen wants a self-checkout system in which a student presents one snack packet to a fixed counter camera. The system compares the captured image with a database of 500 products and retrieves the product name and price. Images may contain rotation, tilt, scale changes, blur, glare, crushed packaging and partial finger occlusion. Similar flavours and packet sizes may have nearly identical designs. The system should achieve at least 95% top-1 accuracy, respond within 1 second and keep stored features within 200 MB on a 4-core CPU PC with 8 GB RAM and no GPU.

Operating Data Used for All Tasks

Parameter	Given value / requirement
Product database	500 products; one clean front-facing colour image per product, 600 × 800 pixels
Counter camera	Fixed USB camera, 1920 × 1080 colour; packet occupies 20–80% of frame
Capture conditions	Rotation up to 45°, tilt up to 30°, distance change up to 4×, mild blur, glare, crushing and partial finger occlusion
Difficult products	Same-brand flavour/size variants; plain packets identified mainly by text
Targets	Top-1 recognition ≥95%; response ≤1 second per packet
Hardware	4 CPU cores, 8 GB RAM, no GPU; Python, OpenCV 4.4+ and NumPy
Memory limit	All stored product features ≤200 MB
Data	At least 30 real snack packets in different conditions; one clean front photo per product

Experimental Setup and Reporting Method

Create a labelled reference database and a separate query set. Capture each packet at multiple rotations, distances, tilts and lighting conditions. Include glossy, folded, partially hidden and visually similar products. Keep product IDs fixed across every experiment. Use a consistent train/reference and test/query split so that near-duplicate images do not leak between sets.

For every method, record preprocessing time, feature extraction time, matching time, total latency, top-1/top-5 accuracy, memory per product and total database memory. Run repeated trials and report mean and standard deviation. Measure on the specified CPU-only PC; do not substitute GPU timings.

Shared Evaluation Measures

Accuracy = (number of correctly recognized query images / total query images) × 100. Precision = TP/(TP+FP), recall = TP/(TP+FN). Top-k accuracy is the fraction for which the correct product occurs within the first k ranked results. Corner localization error is the mean Euclidean distance between predicted and ground-truth corners. Repeatability is the percentage of corresponding points found in both transformed views within the stated tolerance.

For binary descriptor matching use Hamming distance; for SIFT use Euclidean/L2 distance. Lowe's ratio test accepts a nearest match only when d1/d2 is below the selected threshold. RANSAC estimates a homography while rejecting geometrically inconsistent matches. Database memory should include descriptors, keypoint metadata, PCA components/mean and any search index.

Shared Code Pattern: Image Loading, Timing and Metrics

```
import cv2
import numpy as np
import time

img = cv2.imread("packet_query.jpg")
if img is None:
    raise FileNotFoundError("Check the image path")

start = time.perf_counter()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.resize(gray, (640, 360))
elapsed = time.perf_counter() - start

def topk_accuracy(rankings, labels, k=1):
    correct = sum(label in ranking[:k]
                  for ranking, label in zip(rankings, labels))
    return 100.0 * correct / len(labels)

print("Preprocessing time (s):", elapsed)
```

All examples below are implementation templates. Actual accuracy, runtime, memory and recognition claims must be filled using the student's own packet dataset and measured program outputs.

TASK 1 – Types of Image Processing and Their Benefits: Compare Solution Types [4 Marks]

Image processing can be grouped by what it produces: low-level (image in → better image out), mid-level (image in → features out) and high-level (features in → recognition/decision out). Compare three types of solution for recognizing a product at the counter.

Technique	Basic idea and key OpenCV functions
A. Low-level only	Enhance the photo (cv2.cvtColor, cv2.resize, cv2.equalizeHist) and show it to the cashier, who identifies the packet by eye.
B. Low + mid-level	Also extract features (edges, corners, keypoints) and show the 5 most similar products on screen; the student taps the right one.
C. Low + mid + high-level	Extract and match features automatically and bill one identified product with a confidence score.

- Develop and execute three image-based product-billing approaches using Python and OpenCV: (i) low-level image processing, (ii) feature-based recognition, and (iii) high-level product recognition. Process the same packet-image dataset using all three approaches and submit the commented source code with experimentally generated results.
- Analyze the approaches using packet-billing time, billing accuracy, user intervention, consistency across users, and development cost. Present the results in a comparative table.
- Conduct billing experiments with three participants, including yourself, using 10 packets per approach. Measure the time required and record the number of correctly billed packets. Calculate the billing accuracy and substantiate the comparison with experimental evidence.
- Assess the benefits of digital image processing demonstrated by the application in terms of processing speed, repeatability, automation, data storage, and product retrieval. Identify one limitation and discuss its impact on the billing process.
- Analyze the suitability of low-level image processing when a store must bill 300 students within 30 minutes. Examine the consequences of a high-confidence product

misclassification by the high-level recognition system and propose appropriate corrective measures.

- Compare the feature-based approach with a product-recognition model trained on a large image dataset in terms of data requirements, development cost, and model-update effort. Justify the most suitable approach for the given application using measured results and identified constraints.

Evidence Expected: A table classifying each step by type (low / mid / high level), a time and accuracy comparison of the three solution types, and a justified choice.

Ans:

Low-level only

Low-level processing changes image appearance but does not understand the object. Convert the camera image to grayscale or HSV, resize it, correct contrast and display it to a cashier. OpenCV functions include cv2.cvtColor, cv2.resize and cv2.equalizeHist. The human performs the final product identification.

Low + mid-level

After preprocessing, detect edges, corners or local keypoints and compare them with the product database. Display the five closest products so the student or cashier confirms the correct item. This reduces manual browsing but still requires a human decision.

Low + mid + high-level

Extract and match features, rank candidate products, verify geometric consistency and automatically retrieve the product ID and price. A confidence threshold and unknown-product rejection rule are essential before adding an item to the bill.

Comparison and selection

Low-level-only is simple and has low development cost, but it cannot meet high-throughput self-checkout because every item requires visual human inspection. The mid-level method is safer during early deployment because the user confirms a shortlist. The high-level method offers the fastest checkout, but a wrong high-confidence prediction can cause incorrect billing. Use automatic

billing only when both match score and geometric inlier evidence exceed calibrated thresholds; otherwise show the top candidates for confirmation.

Billing capacity calculation

The requested rate is 300 students in 30 minutes = 10 students per minute, or an average of 6 seconds per student. A purely manual visual search can become a bottleneck when each item requires several seconds. A one-second machine recognition target leaves time for scanning multiple items and payment, but actual queue capacity must include payment and user interaction time.

Benefits and limitation

Digital processing is repeatable, fast, easy to store, supports database retrieval and enables automated logs. A limitation is packaging variation: glare, folds or a new design can change features and produce a false match. Maintain an unknown class, ask for confirmation and allow staff correction.

Feature matching versus trained recognition

Feature-based matching can start with a modest labelled reference database and can be updated by adding product images. A trained recognition model may need many labelled examples and retraining or fine-tuning when products change, but may learn more robust semantic cues. Select using measured accuracy, latency, memory and update effort on the actual 500-product catalogue.

Implementation / OpenCV Code Pattern

```
# Low-level display and mid-level ORB shortlist example
import cv2
img = cv2.imread("query.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.resize(gray, (640, 360))
orb = cv2.ORB_create(nfeatures=1000)
kp, des = orb.detectAndCompute(gray, None)
print("Keypoints:", len(kp) if kp is not None else 0)
# High-level stage: rank database descriptors, verify and apply
# a confidence threshold before returning a product ID.
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 2 – Preprocessing: Compare Preprocessing Pipelines [4 Marks]

Before features are extracted, the photo must be prepared so that the same packet gives similar features in every photo. Compare three preprocessing pipelines.

Technique	Basic idea and key OpenCV functions
A. Minimal	Convert to grey level and resize to a fixed width (cv2.cvtColor, cv2.resize).
B. Noise and contrast correction	Pipeline A + Gaussian or median smoothing (cv2.GaussianBlur / cv2.medianBlur) + local contrast enhancement (cv2.createCLAHE).
C. Full normalization	Pipeline B + glare suppression (clip very bright pixels or cv2.inpaint) + crop to the packet region found by edges (Task 4).

- Implement three image preprocessing pipelines using Python and OpenCV on the same set of packet images. Submit the commented source code and ensure that all reported results are obtained through actual program execution.
- Compare the pipelines based on the number of detected keypoints, repeatability of keypoints across two images of the same packet, robustness to blur and glare, and processing time per image. Present the findings in a comparative table.
- Apply an ORB-based feature matcher to 20 image–database pairs after each preprocessing pipeline. Record the number of correct matches and calculate top-1 recognition accuracy using a clearly defined matching criterion and ground-truth labels.
- Assess the effect of resizing a 1920 × 1080 image to different working resolutions on processing time and matching accuracy. Select and justify a suitable working resolution based on the experimental results.

- Investigate the effects of omitting preprocessing while using the full-resolution 1920 × 1080 image and applying excessive smoothing that removes small packet text. Identify the resulting performance limitations and propose appropriate corrective measures.
- Compare preprocessing for deep-learning-based recognition with preprocessing for feature-based matching in terms of resizing, normalization, data augmentation, feature preservation, and computational requirements. Justify the suitability of each preprocessing strategy for its respective recognition approach.

Evidence Expected: Before/after images, a keypoint/accuracy/time table for the three pipelines, and a justified preprocessing pipeline.

Ans:

Pipeline A – Minimal

Convert to grayscale and resize to a fixed working width. It is fast and preserves much of the original information, but it does not correct blur, uneven lighting or glare. It is a useful baseline.

Pipeline B – Noise and contrast correction

Apply Pipeline A, then a small Gaussian or median filter and CLAHE. Gaussian filtering is suitable for mild sensor noise; median filtering is useful for isolated impulse noise. CLAHE improves local contrast and may make printed logos and text more detectable. Excessive smoothing can erase small lettering and excessive CLAHE can amplify glare and texture.

Pipeline C – Full normalization

Apply Pipeline B, detect the packet region using edges/contours, crop it, and suppress isolated glare regions using clipping or inpainting. This gives more consistent packet scale and removes irrelevant background. However, inpainting large shiny regions may invent texture and edge-based cropping may fail on cluttered backgrounds.

Keypoint repeatability

Capture two views of the same packet and match keypoints. A repeatable keypoint should reappear at the corresponding physical location after accounting for rotation and scale. Report number of

keypoints, correct correspondences, repeatability percentage, top-1 accuracy and processing time for all pipelines.

Working resolution

The camera input is 1920 × 1080. Resizing reduces computation approximately with the number of pixels. Compare for example 1280 × 720, 960 × 540 and 640 × 360. Choose the smallest resolution that preserves tiny brand text, flavour labels and enough keypoints to meet the accuracy target.

Failure cases and deep-learning preprocessing

Without preprocessing, noise, shadows and glare may reduce descriptor repeatability. Excessive smoothing removes small packet text that differentiates flavours. Classical feature matching benefits from preserving local gradients and keypoints; deep models usually expect a fixed input size and model-specific normalization, and may use augmentation during training. Do not apply aggressive classical filtering blindly before a learned model.

Implementation / OpenCV Code Pattern

```
img = cv2.imread("query.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
small = cv2.resize(gray, (640, 360))
gauss = cv2.GaussianBlur(small, (3, 3), 0.8)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
enhanced = clahe.apply(gauss)
# A glare mask can be formed from very bright pixels; use inpainting
# only for small isolated glare spots, not large printed regions.
_, glare = cv2.threshold(small, 245, 255, cv2.THRESH_BINARY)
corrected = cv2.inpaint(small, glare, 3, cv2.INPAINT_TELEA)
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 3 – Image Representation: Compare Ways of Representing a Product Packet [4 Marks]

To compare two images, each must be turned into a representation that a computer can compare with a distance or similarity measure. Compare three representations of a product packet.

Technique	Basic idea and key OpenCV functions
A. Raw pixel vector	Resize the packet image to 32 × 32 grey and flatten it to a 1,024-value vector; compare with Euclidean distance.
B. Global histogram	Colour histogram in HSV (cv2.calcHist, cv2.normalize); compare with cv2.compareHist (correlation or chi-square).
C. Local feature set	A set of keypoints with descriptors (e.g., ORB or SIFT); compare by counting good descriptor matches.

- Implement three visual representation techniques for product recognition using Python and OpenCV on the same packet-image dataset. Submit the commented source code and ensure that all reported results are obtained through actual program execution.
- Compare the three representations in terms of storage required per product (bytes), robustness to rotation, scale variation, illumination changes and partial occlusion, ability to distinguish visually similar flavour packets, and product-comparison time.
- For 20 query images, rank all database products using each representation. Calculate and report top-1 and top-5 recognition accuracy using ground-truth product labels. Estimate the

total storage required to represent 500 products and present the results in a comparative table.

- Examine why global image representations may lose discriminative information when part of a packet is occluded by fingers, while local feature representations may retain sufficient visible features for recognition. Support the analysis with experimental observations.
- Assess the limitations of raw-pixel matching when a packet image is rotated by 30° and colour-histogram matching when two flavour variants from the same brand have similar colour distributions. Propose suitable alternatives for each scenario.
- Compare the implemented representations with learned image embeddings extracted from a pretrained CLIP or DINOv2 model. Perform nearest-neighbour product retrieval using the embeddings and evaluate top-1 and top-5 accuracy, storage requirements, and search time. Justify the most suitable representation for the application based on experimental evidence.

Evidence Expected: An accuracy/memory/time table for the three representations, example rankings, and a justified representation.

Ans:

A – Raw pixel vector

Resize each packet to 32 × 32 grayscale and flatten it into 1,024 values. Compare using Euclidean distance. It is compact and simple, but highly sensitive to translation, rotation, scale, illumination and folds. A 30° rotation changes many pixel positions even though the product is unchanged.

B – Global HSV histogram

Calculate normalized colour histograms in HSV and compare with correlation or chi-square distance. Histograms are less sensitive to small spatial shifts and can tolerate some rotation, but discard spatial arrangement. Similar flavour variants from the same brand may have almost identical colour distributions.

C – Local feature set

Store keypoints and descriptors such as ORB or SIFT. Match local visual patterns between query and reference images. Local features can survive partial occlusion because visible regions may still

provide matches. They may fail on plain packets with little texture, strong blur, glare or repeated patterns.

Storage comparison

A raw 1,024-element float32 vector uses 4,096 bytes per product. A normalized 3-channel histogram with 32 bins per channel has 96 bins; float32 storage is 384 bytes per product. Local descriptor storage depends on keypoint count: ORB uses 32 bytes per descriptor, while SIFT uses 128 float32 values (512 bytes) per descriptor, plus keypoint metadata. Multiply by the actual stored feature count for 500 products.

Top-k retrieval

For each of at least 20 query images, rank all database products using the appropriate distance or match score. Report top-1 and top-5 accuracy. Ensure the ground-truth product is not accidentally included as a duplicate query image.

Learned embeddings

CLIP or DINOv2 can produce fixed-length semantic feature vectors. Nearest-neighbour search may be more robust to packaging variation, but inference cost and memory must be measured on the CPU. Compare top-1/top-5 accuracy, vector dimension, storage and search time against the three classical representations.

Implementation / OpenCV Code Pattern

```
# HSV histogram representation
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
hist = cv2.calcHist([hsv], [0, 1, 2], None,
                    [8, 4, 4], [0, 180, 0, 256, 0, 256])
hist = cv2.normalize(hist, hist).flatten()
# Compare two normalized histograms
score = cv2.compareHist(hist1.astype("float32"),
                        hist2.astype("float32"),
                        cv2.HISTCMP_CORREL)
# ORB descriptors are a local-feature representation.
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 4 – Edge Detection: Compare Edge Detectors for Finding the Packet Outline [5 Marks]

Edges are the first features extracted: the outline of the packet is needed to crop and straighten it, and the edges of brand letters are useful features. Compare three edge detectors.

Technique	Basic idea and key OpenCV functions
A. Sobel gradient	First-derivative masks in x and y; gradient magnitude and direction, then a threshold; cv2.Sobel, cv2.magnitude.
B. Laplacian of Gaussian (LoG)	Smooth with a Gaussian, apply the Laplacian, find zero crossings; cv2.GaussianBlur + cv2.Laplacian.
C. Canny	Smoothing, gradient, non-maximum suppression and hysteresis thresholds; cv2.Canny.

- Implement three edge detection techniques using Python and OpenCV on the same set of packet images. Submit the commented source code and ensure that all reported results are obtained through actual program execution.
- Compare the techniques in terms of packet-outline continuity, edge thickness, false edges caused by backgrounds, hands and nearby objects, robustness to blur and glare, and processing time. Present the findings in a comparative table.
- Manually annotate the ground-truth packet outlines in 10 images. Calculate edge precision and recall for each detector using a matching tolerance of 2 pixels. Report the results with appropriate visual evidence.
- Determine suitable Sobel and Canny thresholds using experimental observations. Repeat the evaluation with the selected thresholds halved and doubled, and quantify the resulting changes in edge precision, recall and outline continuity.

- Investigate edge detection under two challenging conditions: (i) packet images captured against crowded, patterned backgrounds and (ii) blurred images processed without prior smoothing. Identify the causes of detection errors and propose suitable corrective measures.
- Compare Canny edge detection with a pretrained learned edge or contour detection model on cluttered counter images containing reflections. Evaluate outline continuity, precision, recall, robustness and processing time, and justify the most suitable approach using experimental evidence.

Evidence Expected: Edge maps, a precision/recall/time table for the three detectors, and a justified detector with thresholds.

Ans:

A – Sobel

Sobel computes first derivatives in x and y. Gradient magnitude highlights intensity transitions and gradient direction describes edge orientation. It is easy to tune and provides directional information, but thresholding may create thick or noisy edges.

B – Laplacian of Gaussian

First smooth with a Gaussian and then compute the second derivative. Zero crossings indicate rapid intensity changes. LoG detects fine detail but can produce responses to texture, printed text and noise; scale selection is important.

C – Canny

Canny smooths the image, computes gradients, applies non-maximum suppression and uses two thresholds with hysteresis. It often produces thin, continuous edges and is a strong baseline for packet outlines, though glare and patterned backgrounds can still cause false edges.

Evaluation

Annotate the true packet outline in 10 images. For a 2-pixel matching tolerance, count predicted edge pixels that match ground truth as true positives; unmatched predictions are false positives and missed ground-truth pixels are false negatives. Calculate precision and recall separately, and report outline continuity and time.

Threshold tuning

Select Sobel and Canny thresholds on a validation subset, then repeat with thresholds halved and doubled. Low thresholds increase recall but may add background edges; high thresholds reduce false positives but may break the packet outline. Keep validation images separate from final test images.

Difficult scenes

Patterned backgrounds and hands introduce competing edges. Use a packet-region prior, background simplification, colour segmentation, contour area/aspect-ratio constraints and a small smoothing kernel. Blur reduces gradient magnitude, so lowering thresholds may help but can increase noise detections.

Learned detector comparison

A learned edge/contour model may handle clutter and reflections better if its training data resembles the counter scene. It may require model weights and more CPU time. Compare precision, recall, outline continuity and latency on identical images; do not assume it is better without measured evidence.

Implementation / OpenCV Code Pattern

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
smooth = cv2.GaussianBlur(gray, (5, 5), 1.0)
sx = cv2.Sobel(smooth, cv2.CV_32F, 1, 0, ksize=3)
sy = cv2.Sobel(smooth, cv2.CV_32F, 0, 1, ksize=3)
mag = cv2.magnitude(sx, sy)
sobel_edges = (mag > 60).astype("uint8") * 255
log = cv2.Laplacian(smooth, cv2.CV_32F, ksize=3)
canny = cv2.Canny(smooth, 50, 150)
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset.

Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 5 – Corner Detection: Compare Harris Corner Detection with Other Corner Detectors [5 Marks]

Corners (where edges meet, such as the ends of letters and the corners of pictures on the packet) are stable points that can be found again in another photo. Compare three corner detectors.

Technique	**Basic idea and key OpenCV functions**
A. Harris corner detector	Structure matrix M from image gradients in a window; corner response $R = \det(M) - k \cdot (\text{trace } M)^2$; cv2.cornerHarris(img, blockSize, ksize, k).
B. Shi–Tomasi (good features to track)	Corner if the smaller eigenvalue of M is large; cv2.goodFeaturesToTrack.
C. FAST	Corner if enough pixels on a circle of 16 pixels are much brighter or darker than the centre; cv2.FastFeatureDetector_create.

- Develop a NumPy-based implementation of the Harris corner detector and validate it against cv2.cornerHarris. Apply all three corner detection techniques to the same packet-image dataset and submit the commented source code. Ensure that all reported results are obtained through actual program execution.
- Comparative Analysis: Compare the detectors in terms of corner repeatability under rotation, scale variation, brightness changes and blur, number and spatial distribution of detected corners, and processing time per image.
- Experimental Validation: Evaluate corner repeatability by rotating packet images by 15° and 45°, scaling them by 0.5× and 2×, and applying brightness changes. Calculate the percentage of corresponding corners detected within a 3-pixel tolerance for each detector and report the results.
- Parameter Analysis: Analyze the eigenvalues of the Harris matrix (M) for flat regions, edges and corners on an actual packet image. Justify the selected values of (k), window size and detection threshold using experimental observations.

- Failure-Scenario Analysis: Assess the limitations of Harris corner detection when packet images are captured from twice the original distance and when an excessively low threshold produces numerous corners around individual letters. Identify suitable corrective measures.
- Comparative Evaluation: Compare Harris corner detection with the learned SuperPoint keypoint detector in terms of repeatability, feature description, robustness to image transformations, computational requirements and suitability for packet recognition. Substantiate the comparison using experimental results.

Evidence Expected: *Corner maps, an eigenvalue analysis, a repeatability/time table for the three detectors, and a justified choice.*

Ans:

A – Harris

Harris uses the structure matrix M formed from image gradients. Its response is $R = \det(M) - k(\text{trace } M)^2$. Large positive responses indicate corners; edges have one dominant eigenvalue and flat regions have two small eigenvalues. Harris is effective for stable corners but is not inherently scale invariant.

B – Shi–Tomasi

Shi–Tomasi selects points based on the smaller eigenvalue of M. It is commonly used for tracking and often provides well-distributed, trackable corners. The number of points and quality threshold must be tuned.

C – FAST

FAST tests a circle of 16 pixels around a candidate centre and declares a corner when a sufficient contiguous arc is much brighter or darker. It is fast and suitable for CPU operation, but may detect many text corners and is sensitive to threshold and scale.

NumPy validation

Compute Ix and Iy, smooth Ix^2, Iy^2 and IxIy over a local window, then calculate the Harris response. Compare the response map with cv2.cornerHarris using the same block size, aperture and k. Differences can arise from derivative scaling, border handling and normalization.

Repeatability experiment

Rotate images by 15° and 45°, scale by 0.5× and 2×, and vary brightness. Map detected corners back to original coordinates using the known transform. Count a correspondence when the distance is within 3 pixels. Report repeatability, number of points and time.

Eigenvalue interpretation and failure

Flat paper has both eigenvalues small; a straight packet edge has one large and one small eigenvalue; a corner has both large. At twice the camera distance, small printed details may disappear and Harris may lose them. An excessively low threshold creates many points around individual letters. Use non-maximum suppression, minimum spacing, scale-aware detection and a region mask.

SuperPoint comparison

SuperPoint learns keypoints and descriptors and may be more repeatable under viewpoint and illumination changes. It has higher model and compute requirements than classical detectors. Compare repeatability, descriptor quality, runtime and memory on the same images.

Implementation / OpenCV Code Pattern

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray32 = np.float32(gray)
R = cv2.cornerHarris(gray32, blockSize=2, ksize=3, k=0.04)
R = cv2.dilate(R, None)
corner_map = np.zeros_like(gray)
corner_map[R > 0.01 * R.max()] = 255

# Shi-Tomasi and FAST
shi = cv2.goodFeaturesToTrack(gray, 300, 0.01, 10)
fast = cv2.FastFeatureDetector_create(threshold=20)
fast_kp = fast.detect(gray, None)
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 6 – Feature Extraction with the Hough Transform: Compare Shape-Finding Methods [5 Marks]

Most snack packets and boxes are roughly rectangular, so its four sides are straight lines. Finding them lets the program straighten (de-skew) the packet before matching. Compare three ways of extracting the packet outline from the edge map of Task 4.

Technique	**Basic idea and key OpenCV functions**
A. Contour approximation	Find the largest closed contour and approximate it by a polygon with four corners; cv2.findContours, cv2.approxPolyDP.
B. Standard Hough line transform	Each edge pixel votes for all lines $\rho = x \cdot \cos\theta + y \cdot \sin\theta$ through it; peaks in the (ρ, θ) accumulator are lines; cv2.HoughLines.
C. Probabilistic Hough line transform	Votes from a random subset of edge pixels; returns line segments with end points; cv2.HoughLinesP.

- Algorithm Formulation: Construct a custom ((\rho,\theta)) accumulator using Python and NumPy for one packet image. Apply all three detection techniques to the same packet-image dataset and submit the commented source code, with all results obtained through actual program execution.
- Performance Assessment: Examine the techniques in terms of detection of all four packet sides under partial occlusion, false lines caused by printed text and counter edges, four-corner localization accuracy, sensitivity to the voting threshold, and processing time.
- Experimental Validation: Measure packet-corner localization error in pixels against manually annotated ground-truth corners for 10 photographs. Apply perspective rectification using cv2.getPerspectiveTransform and determine its effect on product-matching accuracy.
- Accumulator and Circle Detection: Visualize the Hough accumulator and illustrate the mapping of image-space lines to peaks in ((\rho,\theta)) space. Apply cv2.HoughCircles to identify circular logos or price stickers and document the detection results.
- Failure Investigation: Examine the consequences of low voting thresholds that produce false lines and high thresholds that suppress genuine packet boundaries. Determine the effect of

omitting perspective correction before product matching and specify suitable corrective measures.

- Method Selection: Benchmark Hough-based detection against YOLO11 or RT-DETR for packet and bottle detection, including round-lid detection in cluttered scenes. Assess accuracy, robustness, processing time and computational requirements, and substantiate the suitability of each method with experimental evidence.

Evidence Expected: *Accumulator plots, detected outlines, a corner-error/time table for the three methods, and a justified method.*

Ans:

A – Contour approximation

Find external contours from a binary edge map, select a plausible large contour and approximate it with cv2.approxPolyDP. A four-vertex polygon can provide packet corners. It works well for closed outlines but fails when glare, occlusion or shadows break the contour.

B – Standard Hough

Each edge pixel votes for lines in (ρ, θ) space using $\rho = x \cos\theta + y \sin\theta$. Peaks in the accumulator indicate likely lines. It can recover separated collinear edge fragments, but may also select printed text and counter edges.

C – Probabilistic Hough

cv2.HoughLinesP returns finite line segments. It is often more practical for packet sides because endpoints are available and shorter segments can be filtered. Results depend on threshold, minimum line length and maximum gap.

Corner accuracy and rectification

Manually annotate four packet corners in 10 images. Match predicted corners to the correct order and calculate mean Euclidean corner error in pixels. Use cv2.getPerspectiveTransform and cv2.warpPerspective to rectify the packet to a frontal view, then measure whether matching accuracy improves.

Accumulator and circle detection

Visualize the custom (ρ, θ) accumulator: a straight image line maps to a sinusoidal curve, and intersections form peaks. cv2.HoughCircles can detect circular logos or price stickers, but printed circular graphics may create false detections; validate radius and location.

Voting threshold failures

A low threshold accepts weak peaks and creates false lines from text and background; a high threshold misses faint or occluded packet sides. Tune threshold together with minLineLength and maxLineGap. If the packet is not rectangular or is heavily crushed, a quadrilateral model may be inappropriate.

Hough versus object detector

YOLO11 or RT-DETR can detect packet/bottle bounding boxes in clutter, but model inference and training data may exceed the CPU budget. Hough methods are lightweight and geometry-driven. Use detector-based proposals if varied object shapes and overlap make classical geometry unreliable, then compare CPU latency and corner/box accuracy.

Implementation / OpenCV Code Pattern

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 1), 50, 150)
lines = cv2.HoughLinesP(edges, 1, np.pi / 180,
                        threshold=60, minLineLength=80,
                        maxLineGap=15)
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
if contours:
    c = max(contours, key=cv2.contourArea)
    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.02 * peri, True)
    print("Polygon vertices:", len(approx))
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 7 – Scale Invariant Feature Matching: Compare Feature Matching Techniques [5 Marks]

The counter photo and the database image differ in size and rotation, so features must be **scale and rotation invariant**. Compare three keypoint detection and matching techniques.

Technique	**Basic idea and key OpenCV functions**
A. SIFT + brute-force matching + ratio test	Scale-space (difference of Gaussians) keypoints with 128-value descriptors; cv2.SIFT_create, cv2.BFMatcher (L2), knnMatch with Lowe's ratio test.
B. ORB + brute-force Hamming matching	FAST keypoints with orientation and binary BRIEF descriptors; cv2.ORB_create, cv2.BFMatcher(NORM_HAMMING).
C. SIFT + FLANN + RANSAC verification	Fast approximate matching (cv2.FlannBasedMatcher), then keep only matches that agree with one homography (cv2.findHomography with RANSAC).

- Design and Validate three feature-matching techniques using Python and OpenCV on the same packet-image dataset. Submit the commented source code and ensure that all reported results are obtained through actual program execution.
- Performance Assessment: Assess the techniques based on correct matching under scale variation, rotation, viewpoint changes, blur, glare and partial occlusion; discrimination between visually similar flavour packets; descriptor storage size; and comparison time.
- Experimental Validation: Evaluate 30 query photographs against the product database. Report top-1 recognition accuracy, the number of inlier matches for the correct and highest-scoring incorrect products, and processing time. Visualize the matched features using cv2.drawMatches.
- Parameter Optimization: Determine suitable ratio-test thresholds, including 0.6, 0.75 and 0.9, and minimum inlier requirements. Plot correct and incorrect acceptance rates against the selected parameter settings and justify the final configuration using experimental results.
- Failure Analysis: Examine the effects of omitting the ratio test and RANSAC on false matches. Assess the limitations of using the non-invariant Harris corner detector for packet

images captured from twice the original distance, and identify appropriate corrective measures.

- Comparative Evaluation: Benchmark SIFT/ORB-based matching against learned matching approaches, such as SuperPoint with LightGlue or LoFTR, on rotated, blurred and glossy packet images. Assess recognition accuracy, matching robustness, computational cost and suitability for the application using experimental evidence.

Evidence Expected: *Match drawings, an accuracy/inlier/time table for the three techniques, a justified threshold, and a justified choice.*

Ans:

A – SIFT + BF

SIFT detects scale-space keypoints and creates 128-value floating-point descriptors. BFMatcher with L2 distance and Lowe's ratio test removes ambiguous matches. SIFT is robust to scale and rotation and is useful for textured packaging, but descriptor storage and computation are higher than ORB.

B – ORB + BF

ORB combines FAST keypoints with orientation-aware binary descriptors. BFMatcher uses Hamming distance. It is fast and memory-efficient for CPU use, but can be less robust than SIFT under strong viewpoint changes, blur and repeated patterns.

C – SIFT + FLANN + RANSAC

FLANN performs approximate nearest-neighbour matching for floating descriptors. Apply a ratio test, then estimate a homography with RANSAC. Inlier matches agree with a single geometric transformation and are stronger evidence than raw descriptor matches.

Evaluation

Test 30 query photographs against all 500 products. Report top-1 accuracy, correct-product inlier count, highest incorrect-product inlier count and total time. Visualize matches with cv2.drawMatches. Include rotated, blurred, glossy and partially occluded examples.

Parameter optimization

Test ratio thresholds 0.6, 0.75 and 0.9. A strict ratio rejects ambiguous matches and may lower recall; a loose ratio increases accepted matches but can raise false matches. Tune minimum RANSAC inliers and inlier ratio on a validation set. Plot correct and incorrect acceptance rates.

Why ratio test and RANSAC matter

Without the ratio test, repeated logos or similar packet textures can generate ambiguous correspondences. Without RANSAC, unrelated matches may be counted as evidence even when they do not fit one homography. Harris alone is not scale invariant, so it may fail when the packet is photographed from twice the distance.

Learned matching

SuperPoint with LightGlue or LoFTR may provide stronger correspondence under difficult transformations, but model execution can be slow on CPU and memory-heavy. Compare on the same images and hardware. The final choice must meet the one-second limit and distinguish similar flavours.

Implementation / OpenCV Code Pattern

```
sift = cv2.SIFT_create(nfeatures=1500)
kp1, des1 = sift.detectAndCompute(query_gray, None)
kp2, des2 = sift.detectAndCompute(reference_gray, None)
bf = cv2.BFMatcher(cv2.NORM_L2)
pairs = bf.knnMatch(des1, des2, k=2)
good = [m for m, n in pairs if m.distance < 0.75 * n.distance]

if len(good) >= 4:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, inlier_mask = cv2.findHomography(src, dst, cv2.RANSAC, 4.0)
    inliers = int(inlier_mask.sum()) if inlier_mask is not None else 0
    print("Good matches:", len(good), "RANSAC inliers:", inliers)
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 8 – Principal Component Analysis: Compare Feature Reduction Options [4 Marks]

Storing and comparing every feature of 500 products is slow and uses much memory. Principal component analysis (PCA) keeps only the directions with the most variation. Compare three options.

Technique	Basic idea and key OpenCV functions
A. No reduction	Keep full features (e.g., 128-value SIFT descriptors or 1,024-value pixel vectors).
B. PCA on whole-packet pixel vectors ("eigen-packets")	Compute principal components of all 32 × 32 product images and keep k of them; cv2.PCACompute, cv2.PCAProject.
C. PCA on local descriptors	Reduce each SIFT descriptor from 128 to k values (e.g., 32) using PCA learned from database descriptors.

- Formulate and implement three feature-representation options using Python and OpenCV on the same packet-image dataset. Submit the commented source code and ensure that all reported results are obtained through actual program execution.
- Performance Assessment: Assess the three options in terms of memory requirements for 500 products, matching time, top-1 recognition accuracy, and the effect of varying the number of principal components ((k = 8, 16, 32, 64)).
- Experimental Validation: Plot the cumulative explained variance against (k), select the component count using a defined criterion, such as retaining at least 95% of the variance, and tabulate memory consumption, matching time and recognition accuracy for all three options.
- Principal Component Interpretation: Visualize the first few principal components as images for Option B and interpret the visual patterns or image characteristics represented by each component.
- Failure-Scenario Assessment: Examine the effects of selecting too few principal components when distinguishing similar flavour variants from the same brand and training PCA on only

20 products before applying the projection to a database of 500 products. Identify suitable corrective measures.

- Comparative Evaluation: Assess PCA-based compression of handcrafted features against dimensionality reduction of CLIP or DINOv2 embeddings for nearest-neighbour retrieval across 5,000 products. Compare storage requirements, search time, top-1 accuracy and scalability, and justify the findings using experimental evidence.

Evidence Expected: *Explained-variance plot, eigen-packet images, a memory/time/accuracy table for the three options, and a justified k.*

Ans:

A – No reduction

Keep full descriptors or pixel vectors. This preserves all represented information and avoids PCA projection error, but may consume more memory and increase search time. For 500 products, estimate storage from the actual number and data type of features per product.

B – PCA on packet pixels

Resize each packet to 32 × 32 and flatten it into a 1,024-value vector. Learn principal components from the reference images and project each vector onto k components. The components are eigen-packets that capture directions of greatest pixel variation. This can compress global appearance but is sensitive to alignment, rotation and occlusion.

C – PCA on local descriptors

Collect SIFT descriptors from the reference database, learn PCA, and project each 128-dimensional descriptor to k values such as 32. This reduces descriptor memory and matching cost while preserving dominant descriptor variation. It may discard subtle flavour text cues if k is too small.

Explained variance and k

Compute cumulative explained variance and select the smallest k meeting a criterion such as 95%. Test k = 8, 16, 32 and 64. The 95% criterion is a starting point, not a guarantee of best recognition: compare top-1 accuracy as well.

Memory calculation

For N products, D features per product and b bytes per feature element, raw storage is approximately N × D × b, excluding metadata. PCA reduces the feature dimension from d to k, so descriptor storage becomes approximately N × D × k × b. Include the mean vector and component matrix, and report the actual serialized file size.

Principal component interpretation

Reshape the first pixel-PCA component vectors back to 32 × 32 images and display them. Bright/dark regions show image locations that vary strongly across products. Components may encode broad packet colour, logo placement or layout, but they are mathematical variation directions rather than semantic labels.

Failure cases and learned embeddings

Too few components may merge similar flavour variants because small text differences contribute little total variance. Training PCA on only 20 products gives a poor basis for 500 diverse products. Fit PCA on a representative database sample and retain enough components. Compare handcrafted PCA with CLIP/DINOv2 embedding reduction for 5,000-product retrieval using accuracy, memory and search time.

Implementation / OpenCV Code Pattern

```
# PCA on flattened grayscale packet images
X = np.asarray(packet_vectors, dtype=np.float32)  # rows = products
mean, eigenvectors = cv2.PCACompute(X, mean=None, maxComponents=32)
projected = cv2.PCAProject(X, mean, eigenvectors)
# Explained variance can be computed from the eigenvalues of
# the covariance matrix; retain k using cumulative variance.
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 9 – Integrated Self-Checkout: Compare Complete Matching Pipelines [6 Marks]

Join your choices from Tasks 1–8 into one working Self-Checkout and compare three ways of organizing the search over 500 products.

Technique	**Basic idea and key OpenCV functions**
A. Global search only	Preprocess, straighten the packet (Hough), represent it with PCA eigen-packet values, and return the nearest database product.
B. Local search only	Preprocess and match SIFT or ORB features against every database product; return the product with most RANSAC inliers.
C. Two-stage search	Use the fast global representation to shortlist the 10 closest products, then verify only those with local feature matching and RANSAC.

- Integrate three packet-recognition pipelines using Python and OpenCV, and incorporate a fourth pipeline based on zero-shot CLIP recognition using product names as text prompts. Submit the commented source code and ensure that all reported results are obtained through actual program execution.
- Evaluate all four pipelines in terms of top-1 recognition accuracy, processing time per packet against a 1-second requirement, memory consumption against a 200 MB limit, discrimination between visually similar flavour variants, and rejection of products absent from the database.
- Test the four pipelines on the same 50 counter photographs, including glare, finger occlusion, crushed packets, tilted views and blurred images. Compare the measured results with the predictions established in Tasks 1–8 and identify findings that support or contradict those predictions.
- Construct a block diagram covering image acquisition, preprocessing, edge detection, Hough-based outline extraction and perspective correction, corner/keypoint detection, PCA-based feature representation, matching, verification, and product-name and price retrieval for billing. Record the measured processing time for each stage. Prepare a Design Decision Table

- Identify two additional applications that can use the proposed feature-detection and matching framework, such as campus-building recognition in a tour application and medicine-box identification in a pharmacy. Specify the required changes to the image database, recognition criteria and verification process for each application.
- Assess the consequences of automatically billing a product when no reliable database match exists. Identify one experimental result that contradicts the expected theoretical behaviour and explain the discrepancy using the observed image conditions, algorithm limitations or parameter settings.
- Incorporate CLIP-based zero-shot recognition by matching packet images against text descriptions of product names. Evaluate its recognition accuracy on previously unseen products and compare its performance with the other three pipelines in terms of accuracy, processing time, memory consumption and generalization to new products. Substantiate the final system selection with experimental evidence.

Evidence Expected: *A working, commented Self-Checkout, a measured comparison of three pipelines, an original block diagram and decision table, and a defended final design.*

Ans:

A – Global search only

Preprocess and straighten the packet, project its image into PCA eigen-packet space and return the nearest database product. It is compact and fast, but global appearance may confuse similar variants and is sensitive to alignment and occlusion.

B – Local search only

Extract SIFT or ORB descriptors and compare the query against every product. Select the product with the strongest RANSAC inlier evidence. This can be robust to partial occlusion and geometric changes, but exhaustive matching against 500 products may exceed one second.

C – Two-stage search

Use a fast global representation to shortlist the 10 closest products. Perform local feature matching and RANSAC only on these candidates. This balances speed and verification quality and is a strong candidate for the 500-product CPU system.

Fourth pipeline – CLIP zero-shot

Encode the packet image and product-name text prompts using a pretrained CLIP model, then rank text similarities. This may generalize to new product descriptions, but text-only prompts may not distinguish ₹10 and ₹20 variants or near-identical flavours. Use full product descriptions and verify uncertain results visually.

System evaluation

Test all four pipelines on the same 50 counter photographs, including glare, finger occlusion, crushed packets, tilt and blur. Measure top-1 accuracy, latency, memory, similar-variant accuracy and unknown-product rejection. Compare each result with the predictions from Tasks 1–8.

Pipeline block diagram

Camera capture → packet preprocessing → edge/contour detection → Hough outline and perspective correction → keypoint/corner extraction → global PCA shortlist → local descriptor matching → ratio test → RANSAC homography verification → confidence/rejection decision → product ID/name/price lookup → bill update.

Design decision

Prefer two-stage search if measured accuracy reaches 95%, mean or specified percentile latency stays within one second, and stored features remain below 200 MB. If confidence is low or the item is absent from the database, do not bill automatically; display candidates or request cashier intervention.

Additional applications

Campus-building recognition can reuse local descriptors and geometric verification, but needs a building image database captured from multiple viewpoints and robust matching across scale and lighting. Medicine-box identification can reuse packet recognition, but needs stricter SKU/strength/expiry distinctions, barcode or OCR verification and a fail-safe confirmation step.

Unexpected results

A theoretically robust SIFT pipeline may perform poorly on a plain glossy packet because it contains few stable keypoints. ORB may outperform SIFT on simple high-contrast logos due to speed. Explain any discrepancy using texture, glare, blur, repeated patterns, threshold settings and CPU load.

Implementation / OpenCV Code Pattern

```
# Pseudocode for two-stage retrieval
# 1. Compute global descriptor for query and rank all products.
shortlist = rank_global_descriptors(query_image, database, top_k=10)
# 2. Verify only shortlisted candidates with local features.
verified = []
for product_id in shortlist:
    score = local_match_with_ransac(query_image, references[product_id])
    verified.append((product_id, score))
verified.sort(key=lambda item: item[1], reverse=True)
# 3. Bill only if calibrated confidence and inlier checks pass.
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

TASK 10 – Re-analyze and Adapt the Self-Checkout: Multi-Item Recognition and System Adaptation [8 Marks]

Changed Requirement: To shorten queues further, students may now place all their items on a tray (3–10 packets, some overlapping or lying at angles), and the camera must recognize every item in one photo. The store also adds bottles and cans (curved, shiny surfaces) and grows to 5,000 products. The full bill is needed within 2 seconds per tray on the same counter PC with the same 200 MB memory limit.

Technique	**Basic idea and key OpenCV functions**
A. Retain	Run the Task 9 pipeline on the whole tray photo and match it against all 5,000 products, taking the best match as the only item.
B. Modify	Use edges, contours and the Hough transform to separate the items on the tray, then run the Task 9 pipeline on each item.
C. Redesign	Match local features of the whole tray against all products, use RANSAC repeatedly to find each product's group of matches (one homography per item), with PCA-reduced descriptors in a fast index (FLANN).

- Assess the impact of multi-item tray images on the original self-checkout design by identifying invalid assumptions concerning single-item images, rectangular packet geometry, item overlap, database size, processing time and memory requirements. Determine the limitations of using the Task 9 system without modification.
- Quantify the memory requirements and matching time for a database of 5,000 products using the existing feature representations. Measure recognition accuracy on at least 10 self-captured tray photographs and determine the continued suitability of preprocessing, edge detection, corner detection, Hough transforms, SIFT and PCA for multi-item recognition.
- Strategies A, B and C using measured top-1 recognition accuracy, processing time per tray, number of missed items and memory consumption. Formulate a revised self-checkout

pipeline based on the experimental findings and document the changes to the original design.

- Substantiate the final design selection using experimental measurements, memory and processing-time calculations, Task 9 results, and the trade-offs between accuracy, speed and resource consumption. Identify one operating condition under which the selected design would no longer be suitable.
- Examine how few-shot learning or continual learning can support the addition of 50 new products per week without retraining the entire recognition model from scratch. Assess the data requirements, update procedure, computational cost and potential effects on recognition accuracy and previously learned product categories.

Evidence Expected: *Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.*

Ans:

Changed assumptions

The original system assumes one packet fills a substantial part of the frame, has roughly rectangular geometry and can be represented by one homography. A tray image contains several objects, overlap, different orientations and curved reflective surfaces. A single best match cannot produce a complete bill. Database growth from 500 to 5,000 products also increases search and memory requirements.

A – Retain

Running the old pipeline on the entire tray and selecting one best match is unsuitable because it treats the tray as one object and can return only one product. It misses other items and may match a background or one visible package region incorrectly.

B – Modify

Detect edges and contours, separate individual item regions, rectify each packet and run the Task 9 pipeline independently. This is practical for separated rectangular packets, but overlapping objects may merge into one contour; bottles and cans have curved outlines and specular reflections.

C – Redesign

Match local features across the whole tray and repeatedly use RANSAC to find separate geometric groups, with PCA-reduced descriptors and a fast FLANN index. This can recover multiple planar packets, but overlapping items may hide too many features and curved bottles/cans violate a single planar homography.

Evaluation plan

Capture at least 10 tray photographs containing 3–10 products, overlaps, angled packets, bottles and cans. Annotate each item and product ID. For each strategy report per-item recognition accuracy, missed items, duplicate detections, false detections, tray latency, memory and bill completeness.

Memory and search scaling

A 5,000-product database is 10 times larger than 500 products. If per-product feature storage is unchanged, total descriptor storage also grows by approximately 10×. Use measured descriptor counts and file sizes rather than assuming the 200 MB limit will still be met. PCA compression, quantization, product-family indexing and approximate-nearest-neighbour search can reduce storage/search costs.

Revised pipeline

Tray image → detect tray region → multi-object detection/segmentation → assign each item instance → crop and rectify planar packets → curved-object-specific feature or learned detector path → retrieve candidate products using compact indexed descriptors → local verification → deduplicate item detections → unknown/low-confidence review → quantity and price aggregation → final bill.

Final decision

A pure retain strategy should be rejected. Modify may be suitable when objects are separated and mostly rectangular. Redesign is the strongest candidate for overlapping planar packets, but it should be combined with object-instance detection/segmentation for item separation and a separate bottle/can recognition path. Choose only after measured 2-second latency, 200 MB memory and item-level accuracy are verified.

Limitations

The selected design may fail when products are heavily occluded, packets are transparent or highly reflective, or several identical items overlap almost completely. In such cases request repositioning or manual scan rather than guessing.

Few-shot and continual learning

For 50 new products each week, store a small set of labelled reference images and update the feature index without retraining the whole model. Few-shot learning uses a few examples per new class; continual learning incrementally updates a model while attempting to preserve old categories. Monitor old-product accuracy to detect catastrophic forgetting. Version the database and allow rollback.

Implementation / OpenCV Code Pattern

```
# Conceptual per-item processing loop after tray segmentation
for item_crop in detected_item_crops:
    rectified = rectify_if_planar(item_crop)
    candidates = global_index.query(compact_descriptor(rectified), top_k=10)
    product, confidence = verify_local_features(rectified, candidates)
    if confidence >= BILLING_THRESHOLD:
        bill.add(product)
    else:
        bill.request_manual_confirmation(item_crop)
# The functions above represent system stages and must be implemented
# and benchmarked with the actual tray dataset.
```

Evidence to Record

For this task, retain the input/output examples, the exact parameter settings, code execution output and a results table. Record measured values only after running the method on the stated dataset. Compare all three specified techniques under identical conditions and state the reason for the final selection.

OVERALL CONCLUSION

The proposed Smart Self-Checkout combines image preprocessing, edge and corner detection, packet outline extraction, local feature description, dimensionality reduction and geometric feature matching. Global representations can provide a fast shortlist, while local descriptors and RANSAC provide stronger verification. For the original 500-product, single-item scenario, a two-stage retrieval design is a practical candidate because it limits expensive local matching to a small set of products.

For the changed tray scenario, the original single-object pipeline is insufficient. The system must first identify and separate multiple item instances, process each item independently and use separate handling for curved bottles and cans. The final design must be selected using measured item-level accuracy, missed-item rate, latency and memory on the actual CPU hardware.

MARK DISTRIBUTION

Task	Marks
Task 1 – Image Processing Types	4
Task 2 – Image Preprocessing	4
Task 3 – Image Representation	4
Task 4 – Edge Detection	5
Task 5 – Corner Detection	5
Task 6 – Hough Transform	5
Task 7 – Feature Matching	5
Task 8 – PCA-Based Feature Reduction	4
Task 9 – Integrated Self-Checkout Pipeline	6
Task 10 – Multi-Item Recognition and System Adaptation	8
TOTAL	**50**